DEEP LEARNING

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to Deep Learning

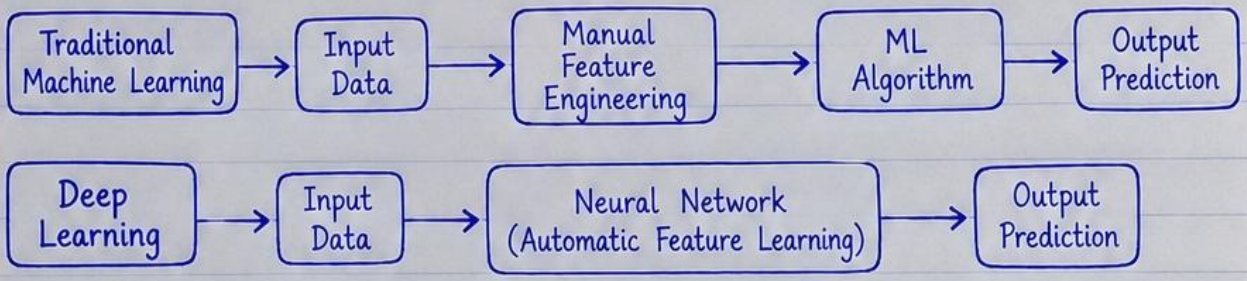
1) What is Deep Learning ?

- Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple layers to learn patterns from large amounts of data.
- It can automatically learn features from raw data (images, text, audio etc.) without manual feature engineering.
- Deep Learning is inspired by the working of the human brain.

2) Why Deep Learning ?

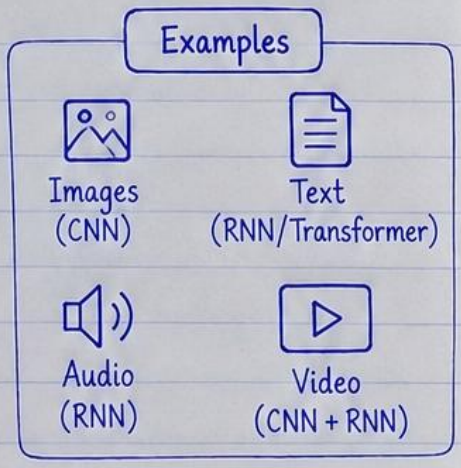
- Works very well on large datasets
- Automatically learns important features
- Handles unstructured data (image, text, audio, video)
- Provides state-of-the-art performance in many real-world tasks like image recognition, speech recognition, NLP, recommendation etc.

3) Traditional ML vs Deep Learning



4) Real-World Applications

- Image Classification (e.g. face detection)
- Object Detection (e.g. self-driving cars)
- Natural Language Processing (e.g. chatbots)
- Speech Recognition (e.g. voice assistants)
- Recommendation Systems (e.g. Netflix, YouTube)
- Medical Image Analysis (e.g. disease detection)
- Time Series Forecasting (e.g. stock market)
- Generative AI (e.g. text, image, video generation)



5) Key Takeaways

- Deep Learning uses neural networks with multiple layers.
- It eliminates manual feature engineering.
- Requires large amount of data and high computational power.
- Widely used in real-world, high-impact applications.

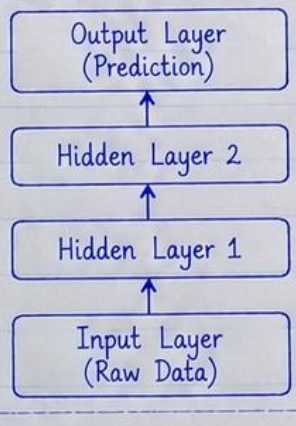
Rahul Yadav

2. How Deep Learning Works ?

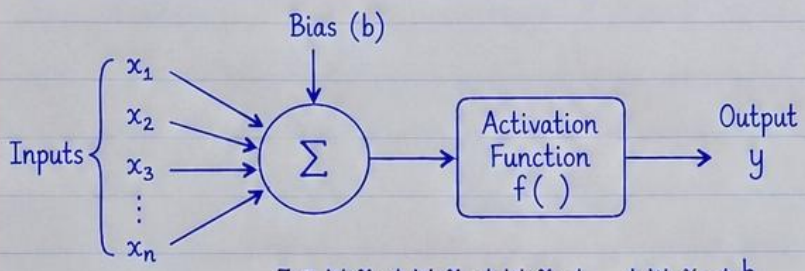
1) Basic Idea

- Deep Learning uses Artificial Neural Networks (ANN) with multiple layers to learn complex patterns from data.
- Each layer learns a different level of representation.
- Lower layers learn simple features (edges, shapes)
- Higher layers learn complex features (objects, concepts)

2) Layered Architecture



3) Neuron (Basic Building Block)



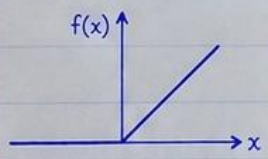
$$z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

$$y = f(z)$$

- Inputs (x) : Features
- Weights (w) : Learnable parameters
- Bias (b) : Shifts the output
- Activation Function (f) : Adds non-linearity
- Output (y) : Passed to next layer

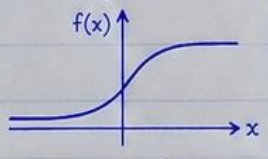
4) Activation Functions

ReLU (Rectified Linear Unit)



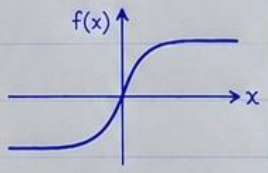
$f(x) = \max(0, x)$

Sigmoid



$$f(x) = \frac{1}{1 + e^{-x}}$$

Tanh



$f(x) = \tanh(x)$

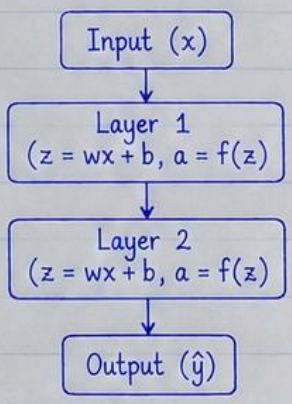
Softmax (for classification)

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j} e^{z_j}}$$

Converts outputs into probabilities (sum = 1)

5) Forward Propagation

- Input data is passed through the network layer by layer.
- At each layer: $z = w \cdot x + b$ (Linear transformation)
- Then activation function is applied: $a = f(z)$
- Output of one layer becomes input to next layer.
- Final layer gives the prediction.



6) Simple Example (1 Neuron)

Suppose $x = 2$, $w = 0.5$, $b = 1$ and ReLU activation

$z = w \cdot x + b = 0.5 \times 2 + 1 = 2$

$y = \text{ReLU}(z) = \max(0, 2) = 2$

Output = 2

Summary:

- Neural networks learn using layers.
- Each neuron performs a linear transformation + activation.
- Activation functions add non-linearity.
- Forward propagation computes the output.

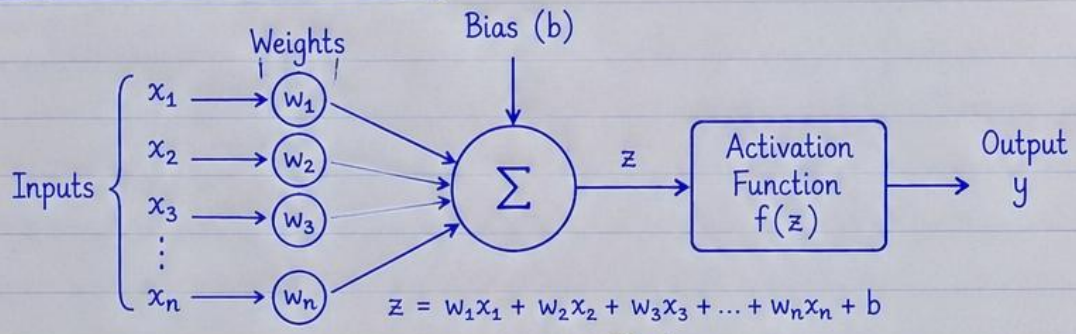
Rahul Yadav

3. Neuron – The Basic Building Block

1) What is a Neuron ?

- A neuron is the basic unit of a neural network.
- It takes multiple inputs, applies weights and bias, performs a mathematical operation and passes the output through an activation function.
- Neurons help the network to learn complex patterns from data.

2) Structure of a Neuron



$$z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n + b$$

$$y = f(z)$$

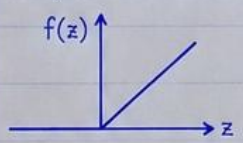
3) Mathematical Representation

- Weighted sum (Linear transformation) : $z = \sum_{i=1}^{n} w_i x_i + b$
- Apply activation function : $y = f(z)$

- x_i : Input features
- w_i : Weights
- b : Bias
- z : Linear combination
- $f(z)$: Activation function
- y : Output of neuron

4) Common Activation Functions

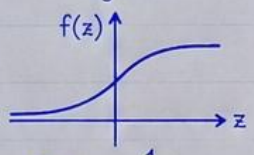
ReLU (Rectified Linear Unit)



$f(z) = \max(0, z)$

- Simple and fast
- Most commonly used

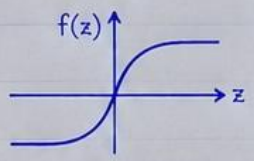
Sigmoid



$f(z) = \frac{1}{1 + e^{-z}}$

- Output between 0 and 1
- Used in binary classification

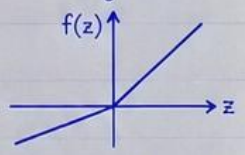
Tanh



$f(z) = \tanh(z)$

- Output between -1 and 1
- Zero centered

Leaky ReLU



$f(z) = \max(\alpha z, z)$

- Solves dying ReLU problem
- α is small (e.g. 0.01)

5) Example Calculation

Suppose a neuron has 3 inputs with ReLU activation.

Inputs : $x_1 = 2,\ x_2 = -1,\ x_3 = 3$

Weights : $w_1 = 0.5,\ w_2 = -0.3,\ w_3 = 0.8$

Bias : $b = 0.1$

Step 1 : Compute z

$z = 0.5 \times 2 + (-0.3) \times (-1) + 0.8 \times 3 + 0.1$

$= 1.0 + 0.3 + 2.4 + 0.1$

$= 3.8$

Step 2 : Apply ReLU

$y = \max(0, 3.8) = 3.8$

Output of neuron = 3.8

6) Key Takeaways

- A neuron performs a linear transformation followed by a non-linear activation function.
- Weights and bias are learnable parameters.
- Activation functions add non-linearity so the network can learn complex patterns.
- Many neurons together form a layer, and multiple layers form a deep neural network.

Rahul Yadav

4. Loss Functions in Deep Learning

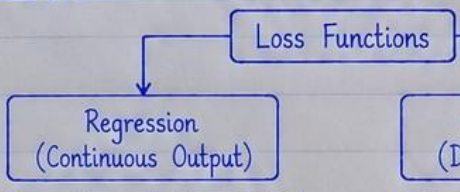
1) What is a Loss Function ?

- A loss function measures how different the model's prediction is from the actual target value.
- It gives a single number (loss) that quantifies the error.
- During training, the model tries to minimize this loss.
- A lower loss means better performance.

2) Why do we need a Loss Function ?

- It helps the model to learn from mistakes.
- The optimization algorithm (e.g. Gradient Descent) uses the loss value to update the weights.
- Without a loss function, the model would not know how good or bad its predictions are.

3) Types of Loss Functions



- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Huber Loss

- Binary Cross Entropy
- Categorical Cross Entropy
- Sparse Categorical Cross Entropy

Key Terms

- y : Actual value (ground truth)
- $\hat{y}$: Predicted value
- n : Number of samples
- C : Number of classes
- L : Loss value
- Lower loss = better model

4) Common Loss Functions with Formulas

(1) Mean Squared Error (MSE)

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Used in regression problems.
- Penalizes large errors more.
- Sensitive to outliers.

(2) Mean Absolute Error (MAE)

$$L = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Used in regression problems.
- Less sensitive to outliers.
- Produces linear error penalty.

(3) Huber Loss

$$L = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{if } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

- Combination of MSE and MAE.
- Less sensitive to outliers.
- Used in robust regression.

(4) Binary Cross Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

- Used for binary classification (2 classes).
- Output $\hat{y}$ is a probability (0 to 1).

(5) Categorical Cross Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

- Used for multi-class classification.
- $y_{i,c}$ is one-hot encoded.
- Measures error across all classes.

(6) Sparse Categorical Cross Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\log(\hat{y}_{i,c})$$

- Used for multi-class classification with integer labels.
- No need for one-hot encoding.
- More memory efficient.

5) Example (Binary Cross Entropy)

Actual value : $y = 1$

Predicted probability : $\hat{y} = 0.8$

$$L = -\left[1 \times \log(0.8) + (1-1) \times \log(1-0.8)\right]$$

$$= -\log(0.8) = 0.223$$

6) Key Takeaways

- Choose the loss function based on the problem type.
- Loss function guides the model to learn.
- Lower loss generally indicates better performance.
- Different loss functions are used for regression and classification.
- Always monitor loss during training to check convergence.

Rahul Yadav

5. Loss Functions

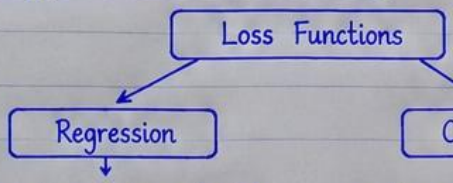
1) What is Loss Function ?

- Loss function measures how far the model's prediction is from the actual target value.
- It gives a single number (loss) that indicates the error.
- During training, the goal is to minimize this loss using optimization algorithms like Gradient Descent.
- A lower loss means better model performance.

2) Why Loss Function is Important ?

- It guides the learning process.
- It helps update model weights in the right direction.
- Different tasks (regression, classification) use different loss functions.

3) Types of Loss Functions



- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Huber Loss

- Binary Cross Entropy
- Categorical Cross Entropy
- Sparse Categorical Cross Entropy

4) General Formula

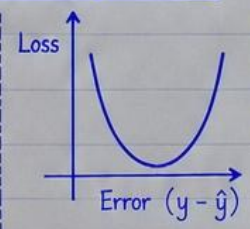
$$\text{Loss} = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \hat{y}_i)$$

- n : Number of samples
- y_i : Actual value
- $\hat{y}_i$: Predicted value
- $L(\,)$: Loss function
- Loss : Average error over all samples

5) Mean Squared Error (MSE) - For Regression

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- Measures the average of squared differences.
- Penalizes larger errors more.
- Widely used for regression problems.



Example:

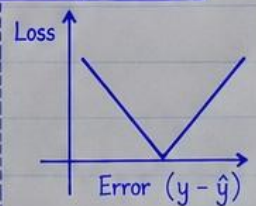
Actual (y) : [3, 5, 2]
Predicted (ŷ): [2, 5, 4]

$$MSE = \frac{1}{3}\left[(3-2)^2 + (5-5)^2 + (2-4)^2\right]$$
$$= \frac{1}{3}\left[1 + 0 + 4\right]$$
$$= 1.67$$

6) Mean Absolute Error (MAE) - For Regression

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

- Measures the average of absolute differences.
- Less sensitive to outliers compared to MSE.



Example:

Actual (y) : [3, 5, 2]
Predicted (ŷ): [2, 5, 4]

$$MAE = \frac{1}{3}\left[|3-2| + |5-5| + |2-4|\right]$$
$$= \frac{1}{3}\left[1 + 0 + 2\right] = 1.0$$

7) Binary Cross Entropy - For Binary Classification

$$L = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

- Used when output is a probability (0 or 1).
- Ideal for binary classification (Yes/No).
- Lower loss means predicted probability is closer to actual label.

8) Categorical Cross Entropy - For Multi-class

$$L = -\frac{1}{n} \sum_{i=1}^{n} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c})$$

- Used for multi-class classification.
- $y_{i,c}$: Actual label (one-hot encoded)
- $\hat{y}_{i,c}$: Predicted probability for class c
- C : Total number of classes

9) Key Takeaways

- Loss function defines how well the model is performing.
- Choose the right loss based on the problem (regression or classification).
- During training, we minimize the loss using optimization algorithms.
- Lower loss leads to better predictions.

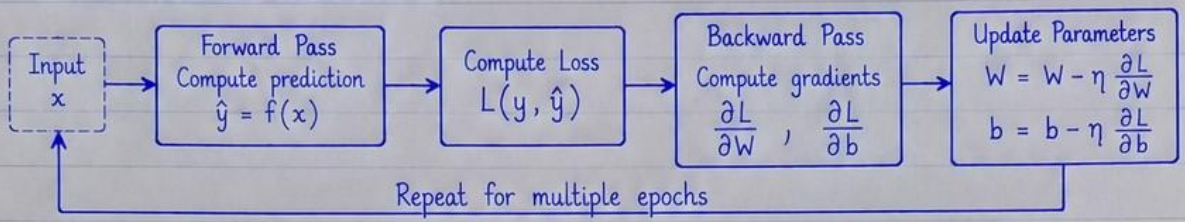
Rahul Yadav

6. Backpropagation

1) What is Backpropagation ?

- Backpropagation is an algorithm used to compute the gradients of the loss function with respect to the model's parameters (weights and biases).
- It uses the chain rule of calculus to efficiently compute gradients layer by layer from output to input.
- These gradients are then used by an optimization algorithm (such as Gradient Descent) to update the parameters.

2) Backpropagation Flow



3) Chain Rule

- Backpropagation uses the chain rule to compute gradients.
- If $z = f(g(x))$, then

$$\frac{dz}{dx} = \frac{dz}{dg} \times \frac{dg}{dx}$$

4) Example: Single Neuron

Let's consider a single neuron with MSE loss and linear activation.

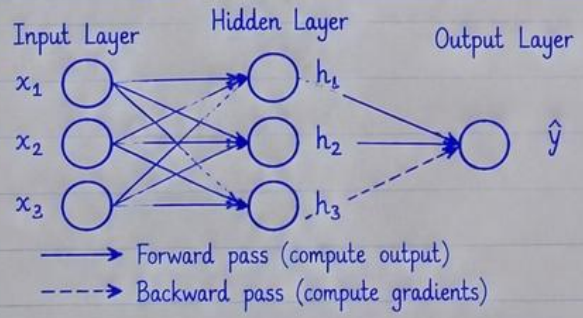
$$\hat{y} = wx + b \qquad L = (y - \hat{y})^2$$

- Gradient w.r.t weights and bias :

$$\frac{\partial L}{\partial w} = 2(y - \hat{y})(-x) = -2x(y - \hat{y})$$

$$\frac{\partial L}{\partial b} = 2(y - \hat{y})(-1) = -2(y - \hat{y})$$

5) Backpropagation in a Neural Network



Steps in Backpropagation:

1. Perform forward propagation to compute $\hat{y}$.
2. Compute the loss $L(y, \hat{y})$.
3. Compute gradients of loss w.r.t output layer.
4. Propagate gradients backward through hidden layers using chain rule.
5. Update weights and biases using an optimizer (e.g., Gradient Descent).

6) Weight Update Rule (Gradient Descent)

$$W = W - \eta \frac{\partial L}{\partial W} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

- η : Learning rate (controls step size)
- $\frac{\partial L}{\partial W}$: Gradient of loss w.r.t weights
- $\frac{\partial L}{\partial b}$: Gradient of loss w.r.t bias

7) Backpropagation with Activation Function

For a neuron : $z = w \cdot x + b$, $a = f(z)$

Gradient of loss w.r.t z :

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \times \frac{\partial a}{\partial z} \quad \text{(Chain rule)}$$

Examples :

- For ReLU : $\frac{\partial a}{\partial z} = 1$ (if $z > 0$) else 0
- For Sigmoid : $\frac{\partial a}{\partial z} = \sigma(z)(1 - \sigma(z))$
- For Tanh : $\frac{\partial a}{\partial z} = 1 - \tanh^2(z)$

8) Key Takeaways

- Backpropagation computes gradients efficiently using chain rule.
- Gradients tell how much each parameter contributes to the loss.
- Parameters are updated using optimization algorithms.
- It is the core algorithm that allows neural networks to learn.

Rahul Yadav

7. Optimization Algorithms

1) Why Optimization is Important ?

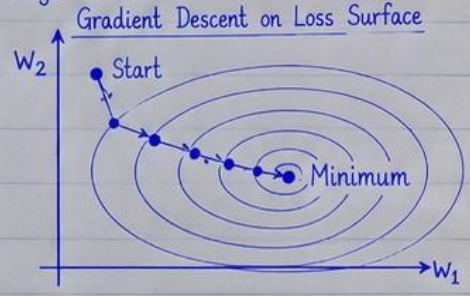
- Optimization algorithms are used to update the model parameters (weights and biases) to minimize the loss function.
- They help the model learn from data by finding the best parameters.
- The choice of optimizer can affect the training speed and final performance.
- Different optimizers work better for different types of problems.

2) Gradient Descent (GD)

- Basic optimization algorithm used to minimize the loss function.
- It updates parameters in the opposite direction of the gradient.
- Works well for small datasets but can be slow for large datasets.

$$\theta = \theta - \eta \nabla L(\theta)$$

- θ : Model parameters (weights, biases)
- η : Learning rate
- $\nabla L(\theta)$: Gradient of loss w.r.t parameters



3) Types of Gradient Descent

Batch Gradient Descent

- Uses the entire training data to compute the gradient.
- More stable but slower.
- Good for small datasets.

$$\theta = \theta - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla L_i(\theta)$$

Stochastic Gradient Descent (SGD)

- Uses one training example at a time.
- Faster but more noisy.
- Can escape local minima.

$$\theta = \theta - \eta \nabla L_i(\theta)$$

Mini-Batch Gradient Descent

- Uses a small batch of training examples.
- Combines benefits of batch and stochastic GD.
- Most commonly used in practice.

$$\theta = \theta - \eta \frac{1}{b} \sum_{i=1}^{b} \nabla L_i(\theta)$$

4) Popular Optimization Algorithms

Algorithm	Key Idea	Pros	Cons	Use Cases
SGD	Basic gradient descent	Simple, fast	Noisy, slow convergence	Small datasets
Momentum	Uses past gradients to accelerate	Faster convergence, reduces oscillations	Extra hyperparameter (β)	Most deep learning tasks
RMSprop	Adaptive learning rate using moving average	Works well for non-stationary problems	Can be sensitive to hyperparameters	RNN, noisy data
Adam	Combines Momentum + RMSprop	Fast, adaptive, works in most cases	More memory, may not generalize well always	Default choice for most deep learning models

5) Adam Optimizer (Most Popular)

- Adam (Adaptive Moment Estimation) combines ideas of Momentum and RMSprop.
- It maintains two moving averages:
  - ① First moment (mean of gradients) m_t
  - ② Second moment (variance of gradients) v_t

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta = \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- g_t : Gradient at step t
- β_1 : Decay rate for first moment (usually 0.9)
- β_2 : Decay rate for second moment (usually 0.999)
- η : Learning rate (e.g. 0.001)
- ϵ : Small value to prevent division by zero (e.g. 10^{-8})

6) Key Takeaways

- Optimization algorithms update model parameters to minimize the loss.
- Adam is the most popular optimizer in deep learning.
- Learning rate and other hyperparameters significantly affect performance.
- Different optimizers may work better for different problems, so experimentation is important.

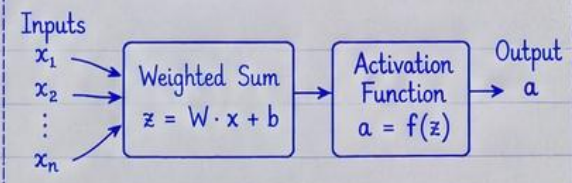
Rahul Yadav

8. Activation Functions

1) What is Activation Function ?

- Activation function introduces non-linearity into the neural network.
- It decides whether a neuron should be activated or not.
- Without activation function, the whole network would behave like a linear model.
- It helps the model learn complex patterns in the data.

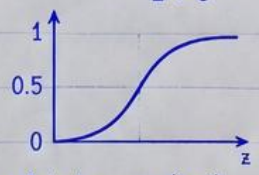
Role in a Neuron



2) Common Activation Functions

1. Sigmoid Function

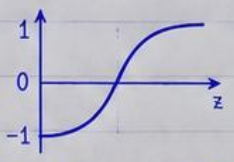
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Output range : (0, 1)
- Used in binary classification
- Can suffer from vanishing gradient problem

2. Tanh Function

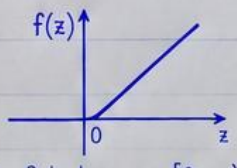
$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



- Output range : (-1, 1)
- Zero centered
- Can also suffer from vanishing gradient

3. ReLU Function

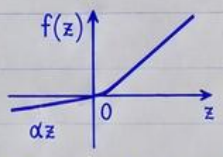
$$f(z) = \max(0, z)$$



- Output range : [0, ∞)
- Most widely used
- Simple and computationally efficient
- Can cause "dying ReLU" problem

4. Leaky ReLU Function

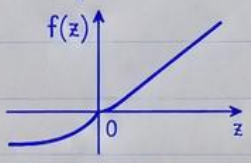
$$f(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}$$



- Allows small gradient for negative values
- Solves dying ReLU problem
- α is a small value (e.g., 0.01)

5. ELU (Exponential Linear Unit)

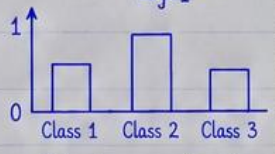
$$f(z) = \begin{cases} z & \text{if } z \geq 0 \\ \alpha(e^{z} - 1) & \text{if } z \leq 0 \end{cases}$$



- Smooth curve for negative values
- Helps faster learning
- Less prone to dying neurons

6. Softmax Function

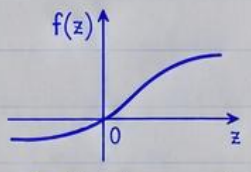
$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$



- Output range : (0, 1)
- Sum of all outputs = 1
- Used in multi-class classification

7. Swish Function

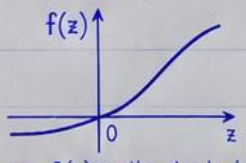
$$f(z) = z \cdot \sigma(z)$$



- Smooth and non-monotonic
- Often performs better than ReLU
- Used in modern architectures

8. GELU Function

$$f(z) = z \cdot \Phi(z)$$



- $\Phi(z)$ is the standard normal CDF
- Used in transformer models (BERT, GPT)
- Works well in practice

3) Comparison of Activation Functions

Function	Output Range	Zero Centered	Vanishing Gradient	Common Use Case
Sigmoid	(0, 1)	No	Yes	Binary classification
Tanh	(-1, 1)	Yes	Yes	General purpose
ReLU	[0, ∞)	No	No	Hidden layers (most used)
Leaky ReLU	(-∞, ∞)	No	No	When dying ReLU is a problem
ELU	(-α, ∞)	No	No	Better performance
Softmax	(0, 1)	No	-	Multi-class classification (output layer)
Swish	(-∞, ∞)	No	No	Modern deep learning models
GELU	(-∞, ∞)	No	No	Transformers (BERT, GPT)

4) Key Takeaways

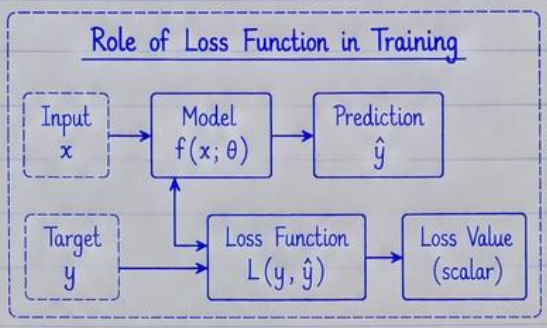
- Activation functions introduce non-linearity in neural networks.
- ReLU and its variants are most commonly used in hidden layers.
- Sigmoid and tanh are mainly used in output layers or specific problems.
- Softmax is used for multi-class classification.
- The right activation function can improve training speed and model performance.

Rahul Yadav

9. Loss Functions

1) What is a Loss Function ?

- A loss function measures the difference between the predicted output and the actual target value.
- It tells us how well the model is performing.
- The goal of training is to minimize the loss function by updating the model parameters using optimization algorithms (e.g., Gradient Descent).
- Different loss functions are used for different types of problems (regression, classification, etc.).



2) Types of Loss Functions

- Regression Loss Functions (e.g., MSE, MAE, Huber Loss)
- Classification Loss Functions (e.g., Binary Cross-Entropy, Categorical Cross-Entropy)
- Other Loss Functions (e.g., Hinge Loss, KL Divergence)

3) Regression Loss Functions

a) Mean Squared Error (MSE)

$$L = \frac{1}{n}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

- Measures the average of squared differences.
- Penalizes larger errors more.
- Widely used for regression problems.

b) Mean Absolute Error (MAE)

$$L = \frac{1}{n}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

- Measures the average of absolute differences.
- Less sensitive to outliers compared to MSE.
- Useful when we want robust predictions.

c) Huber Loss

$$L = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{if } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

- Combination of MSE and MAE.
- Less sensitive to outliers.
- Used in regression problems with noisy data.

4) Classification Loss Functions

a) Binary Cross-Entropy (Log Loss)

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

- Used for binary classification (0 or 1).
- Measures the difference between actual label and predicted probability.
- Widely used in logistic regression, neural networks, etc.

b) Categorical Cross-Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik})$$

- Used for multi-class classification.
- y_{ik} is 1 if class k is the correct class, else 0.
- $\hat{y}_{ik}$ is the predicted probability for class k.
- Commonly used with softmax activation.

c) Sparse Categorical Cross-Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\log(\hat{y}_{i\,y_i})$$

- Used for multi-class classification when labels are given as integers (e.g., 0, 1, 2, ...).
- Simpler than categorical cross-entropy because we use class index directly.

d) Hinge Loss

$$L = \frac{1}{n}\sum_{i=1}^{n}\max(0, 1 - y_i \cdot \hat{y}_i)$$

- Used in Support Vector Machines (SVM).
- $y_i \in \{-1, 1\}$.
- Encourages correct classification with margin.

e) Kullback-Leibler Divergence (KL Divergence)

$$L = \sum_{i=1}^{n} y_i \log\left(\frac{y_i}{\hat{y}_i}\right)$$

- Measures the difference between two distributions.
- Often used in generative models and knowledge distillation.
- y_i is the true distribution and $\hat{y}_i$ is the predicted distribution.

f) Focal Loss (for Imbalanced Data)

$$L = -\frac{1}{n}\sum_{i=1}^{n}\alpha(1-\hat{y}_i)^{\gamma}\log(\hat{y}_i)$$

- Variant of cross-entropy.
- Reduces the impact of easy examples and focuses on hard examples.
- Useful for imbalanced classification problems (e.g., fraud detection).

5) Key Takeaways

- Loss function quantifies the difference between prediction and actual value.
- Different loss functions are used for different types of problems.
- The choice of loss function affects the model's performance and convergence speed.
- During training, we minimize the loss function using optimization algorithms like Gradient Descent.
- Understanding loss functions is essential for building effective machine learning and deep learning models.

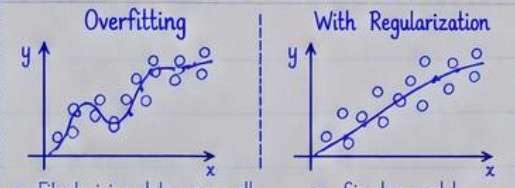
Rahul Yadav

10. Regularization Techniques

1) Why Regularization ?

- Regularization is used to prevent overfitting in machine learning models.
- It adds a penalty term to the loss function.
- It helps the model generalize better to unseen data.
- It is especially useful when the model is too complex or when we have high-dimensional data.
- Common regularization techniques are L1 (Lasso), L2 (Ridge) and Elastic Net.

Overfitting vs Regularization



- Fits training data very well
- Poor performance on new data
- Simpler model
- Better generalization

2) Regularized Loss Function

General form of a regularized loss function:

$$L(\theta) = L_{data}(\theta) + \lambda \cdot R(\theta)$$

- $L(\theta)$: Regularized loss function
- $L_{data}(\theta)$: Original loss function (e.g., MSE)
- $R(\theta)$: Regularization term
- λ : Regularization parameter (controls strength)
- θ : Model parameters (weights)

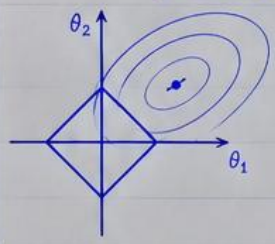
3) L1 Regularization (Lasso)

Loss function with L1 regularization:

$$L(\theta) = L_{data}(\theta) + \lambda \sum_{i=1}^{n} |\theta_i|$$

- Adds absolute value of parameters to the loss.
- Encourages sparsity (some weights become zero).
- Useful for feature selection.
- Works well when we have many irrelevant features.
- Commonly used in linear regression and logistic regression.

Effect of L1 Regularization



- Diamond shape constraint (L1 norm)
- Can set some weights exactly to zero
- Performs feature selection

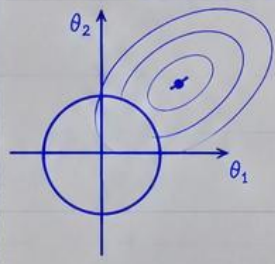
4) L2 Regularization (Ridge)

Loss function with L2 regularization:

$$L(\theta) = L_{data}(\theta) + \lambda \sum_{i=1}^{n} \theta_i^2$$

- Adds squared value of parameters to the loss.
- Shrinks the weights but does not make them exactly zero.
- Reduces model complexity and prevents overfitting.
- Works well when features are correlated.
- Commonly used in linear regression, logistic regression and neural networks.

Effect of L2 Regularization



- Circular shape constraint (L2 norm)
- Shrinks all weights towards zero
- Does not perform feature selection
- More stable when features are correlated

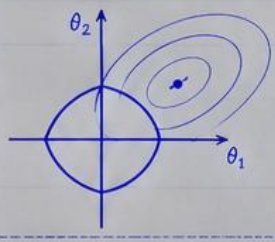
5) Elastic Net Regularization

Combines both L1 and L2 regularization:

$$L(\theta) = L_{data}(\theta) + \lambda_1 \sum_{i=1}^{n} |\theta_i| + \lambda_2 \sum_{i=1}^{n} \theta_i^2$$

- Helps when we have high-dimensional data with correlated features.
- Provides a balance between sparsity and stability.
- Often performs better than using L1 or L2 alone.

Effect of Elastic Net



- Combination of L1 and L2
- Can perform feature selection
- More stable with correlated features

6) Choosing the Regularization Parameter (λ)

- λ controls the strength of regularization.
- Small λ → less regularization (may overfit)
- Large λ → more regularization (may underfit)
- Use techniques like Cross-Validation to find the best value of λ.

Effect of Different λ Values

λ Value	Model Complexity	Bias	Variance	Result
Very small	High	Low	High	Overfitting
Optimal	Balanced	Balanced	Balanced	Good generalization
Very large	Low	High	Low	Underfitting

7) Key Takeaways

- Regularization helps prevent overfitting by adding a penalty term to the loss function.
- L1 makes weights sparse, L2 shrinks weights, and Elastic Net combines both.
- The regularization parameter (λ) is important and should be tuned using cross-validation.
- Regularization is widely used in regression, classification and deep learning models.

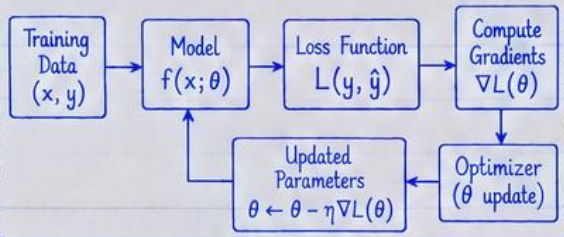
Rahul Yadav

11. Optimization in Deep Learning

1) What is Optimization ?

- Optimization is the process of finding the best model parameters (weights and biases) that minimize the loss function.
- It helps the model learn from data and improve performance.
- Different optimization algorithms update the parameters in different ways.
- A good optimizer can lead to faster convergence and better generalization.

Overview of the Optimization Process



2) Gradient Descent (Recap)

- Gradient Descent is the basic optimization algorithm.
- It updates parameters in the opposite direction of the gradient to minimize the loss.
- Update rule :

$$\theta = \theta - \eta \nabla L(\theta)$$

- θ : Model parameters (weights, biases)
- η : Learning rate (step size)
- $\nabla L(\theta)$: Gradient of loss w.r.t parameters

3) Types of Optimization Algorithms

a) Batch Gradient Descent

- Uses the entire training data to compute the gradient.
- Stable but slow.
- Good for small datasets.

$$\theta = \theta - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla L_i(\theta)$$



Smooth convergence (but slow)

b) Stochastic Gradient Descent (SGD)

- Uses one training example at a time.
- Faster but noisy.
- Can escape local minima.

$$\theta = \theta - \eta \nabla L_i(\theta)$$



Noisy updates (faster, may fluctuate)

c) Mini-Batch Gradient Descent

- Uses a small batch of training examples.
- Combines benefits of batch and stochastic GD.
- Most commonly used.

$$\theta = \theta - \eta \frac{1}{b} \sum_{i=1}^{b} \nabla L_i(\theta)$$



Good balance (fast and stable)

d) Adaptive Optimizers

- Automatically adjust the learning rate for each parameter.
- Examples: Momentum, RMSprop, Adam, etc.
- Work well for complex problems and deep neural networks.

4) Popular Optimization Algorithms

Algorithm	Key Idea	Pros	Cons	Common Use Cases
SGD	Basic gradient descent	Simple, low memory	Slow convergence, noisy	Small datasets, baseline training
Momentum	Uses past gradients (velocity)	Faster convergence, reduces oscillations	Requires extra hyperparameter (β)	Deep networks, recurrent models
RMSprop	Adaptive learning rate using moving average of squared gradients	Works well for non-stationary problems	May not generalize well in some cases	RNN, noisy data
Adam	Combines Momentum + RMSprop	Fast, adaptive, works in most cases	May not always generalize well	Default choice for most deep learning models

5) Adam Optimizer (Most Popular)

- Adam (Adaptive Moment Estimation) combines the ideas of Momentum and RMSprop.
- It maintains two moving averages:
  1. First moment (mean of gradients) m_t
  2. Second moment (variance of gradients) v_t

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta = \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- g_t : Gradient at step t
- β_1 : Decay rate for first moment (usually 0.9)
- β_2 : Decay rate for second moment (usually 0.999)
- η : Learning rate (e.g. 0.001)
- ϵ : Small value to prevent division by zero (e.g. 10^{-8})

6) Key Takeaways

- Optimization helps prevent overfitting by adding a penalty term to the loss function.
- L1, L2 and Elastic Net regularization are commonly used techniques.
- The choice of optimization algorithm and its hyperparameters affect the model's performance.
- Experimentation and cross-validation are important to find the best optimizer and settings.

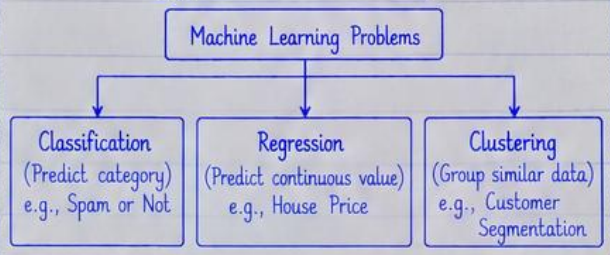
Rahul Yadav

12. Evaluation Metrics in Machine Learning

1) Why Evaluation Metrics ?

- Evaluation metrics help us measure how well the model is performing.
- They give a clear idea about the model's accuracy and generalization capability.
- Different metrics are used for different types of problems (classification, regression, etc.).
- Using the right metric is important for choosing the best model.

Types of ML Problems



2) Confusion Matrix (for Classification)

Actual	Predicted	
	Positive	Negative
Positive	TP (True Positive)	FN (False Negative)
Negative	FP (False Positive)	TN (True Negative)

- TP: Predicted positive and actually positive
- TN: Predicted negative and actually negative
- FP: Predicted positive but actually negative (Type I error)
- FN: Predicted negative but actually positive (Type II error)

3) Classification Evaluation Metrics

a) Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Most common metric.
- Works well when data is balanced.
- Can be misleading for imbalanced data.

b) Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Measures how many predicted positives are actually correct.
- High precision → less false positives.

c) Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Measures how many actual positives are correctly identified.
- High recall → less false negatives.

d) F1-Score

$$F1 = 2 \times \left\{ \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right\}$$

- Harmonic mean of precision and recall.
- Useful when we need balance between precision and recall.

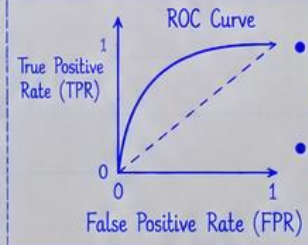
4) Other Classification Metrics

a) Specificity

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- Measures how many actual negatives are correctly identified.
- Useful in medical diagnosis (e.g., disease detection).

b) ROC Curve and AUC



- Shows the trade-off between TPR and FPR at different thresholds.
- AUC (Area Under Curve) close to 1 is better.

c) Log Loss (Cross-Entropy Loss)

$$L = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1-y_i)\log(1-\hat{p}_i) \right]$$

- Measures the performance of probability predictions.
- Lower log loss is better.
- Widely used in classification models like Logistic Regression, Neural Networks.

5) Regression Evaluation Metrics

a) Mean Squared Error (MSE)

$$MSE = \frac{1}{n}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

- Measures average of squared errors.
- Penalizes larger errors more.
- Widely used for regression problems.

b) Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

- Measures average of absolute errors.
- Less sensitive to outliers compared to MSE.
- Easier to interpret.

c) Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- Square root of MSE.
- In the same unit as the target variable.
- Commonly used for regression evaluation.

d) R^2 Score (Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

- Explains the proportion of variance in the target variable.
- Range: $(-\infty, 1]$
- Closer to 1 is better.

6) Key Takeaways

- Use classification metrics like Accuracy, Precision, Recall, F1-Score, ROC-AUC based on the problem and data imbalance.
- Use regression metrics like MSE, MAE, RMSE and R^2 for continuous prediction problems.
- There is no single best metric → choose the metric based on your business problem and data characteristics.
- Always evaluate the model on unseen test data to check its generalization.
- Combine multiple metrics to get a complete picture of model performance.

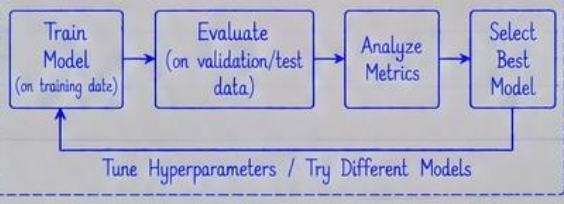
Rahul Yadav

13. Model Evaluation and Validation

1) Why Model Evaluation and Validation ?

- Evaluation helps us understand how well the model performs on unseen data.
- Validation ensures that the model generalizes well and is not overfitting.
- It helps in selecting the best model and tuning hyperparameters.
- Different evaluation techniques are used for different problems and datasets.

Model Evaluation Workflow



2) Train, Validation and Test Split

- Training Set: Used to train the model.
- Validation Set: Used to tune hyperparameters and compare models.
- Test Set: Used to evaluate the final performance on unseen data.

Training (70%)	Validation (15%)	Test (15%)

3) Evaluation Metrics for Classification

a) Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Measures overall correct predictions.
- Good when classes are balanced.

b) Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Measures how many predicted positives are actually correct.
- High precision → less false positives.

c) Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Measures how many actual positives are correctly identified.
- High recall → less false negatives.

d) F1-Score

$$F1 = 2 \times \left\{ \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right\}$$

- Harmonic mean of precision and recall.
- Useful when we need a balance between precision and recall.

4) Confusion Matrix

Actual	Predicted	
	Positive	Negative
Positive	TP (True Positive)	FN (False Negative)
Negative	FP (False Positive)	TN (True Negative)

Binary Classification Metrics

- TP : Predicted positive and actually positive
- TN : Predicted negative and actually negative
- FP : Predicted positive but actually negative (Type I error)
- FN : Predicted negative but actually positive (Type II error)

5) Evaluation Metrics for Regression

a) Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- Measures average of squared errors.
- Penalizes larger errors more.
- Widely used for regression problems.

b) Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

- Measures average of absolute errors.
- Less sensitive to outliers compared to MSE.
- Easier to interpret.

c) Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

- Square root of MSE.
- In the same unit as the target variable.
- Commonly used for regression evaluation.

d) R^2 Score (Coefficient of Determination)

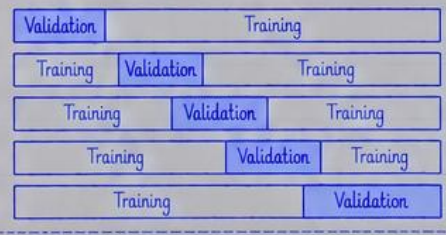
$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

- Explains the proportion of variance in the target variable.
- Range: $(-\infty, 1]$
- Closer to 1 is better.

6) Cross-Validation

- Cross-validation helps in better estimation of model performance by using multiple train-validation splits.
- K-Fold Cross-Validation is the most common technique.
- It reduces the risk of overfitting and gives a more reliable performance estimate.
- Common values of K: 5 or 10.

K-Fold Cross-Validation (K = 5)



7) Key Takeaways

- Use appropriate metrics based on problem type and business need.
- Always evaluate on unseen data (validation/test set).
- Use cross-validation for reliable performance estimation.
- Compare multiple models and choose the one with the best metrics.
- Consider both evaluation metrics and real-world constraints.

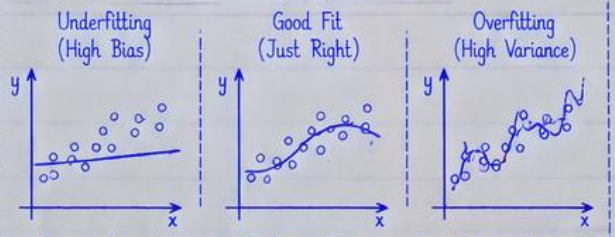
Rahul Yadav

14. Overfitting and Underfitting

1) What are Overfitting and Underfitting?

- Overfitting and underfitting are common problems in machine learning models.
- Underfitting happens when the model is too simple to learn the pattern in data.
- Overfitting happens when the model is too complex and learns noise instead of the actual pattern.
- The goal is to find the right balance so the model generalizes well to new data.

Visual Comparison



- Too simple model
- Does not capture pattern

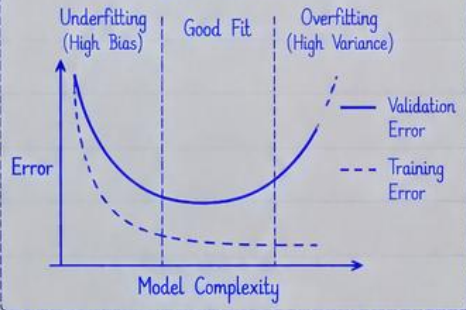
- Captures the pattern
- Good generalization

- Too complex model
- Learns noise

2) Characteristics

Aspect	Underfitting (High Bias)	Overfitting (High Variance)
Model Complexity	Too simple	Too complex
Training Error	High	Very low
Validation/Test Error	High	High
Generalization	Poor	Poor
Learns	Fails to learn pattern	Learns noise in training data
Examples	Linear model for complex data	Very deep neural network on small data

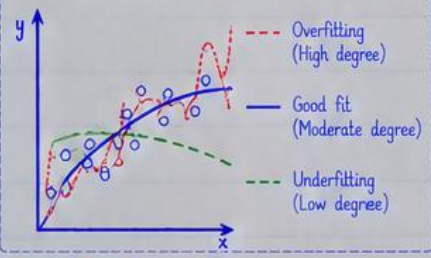
Training vs Validation Error



3) Reasons

Reasons for Underfitting	Reasons for Overfitting
• Model is too simple. • Not enough features. • Important patterns are missing. • High regularization. • Insufficient training time. • Using a linear model for non-linear data.	• Model is too complex. • Too many features (including irrelevant ones). • Very low regularization. • Training data is small. • Model learns noise and outliers. • Training for too many epochs.

Example: Polynomial Regression



4) How to Fix Underfitting?

- Increase model complexity (more parameters).
- Add more relevant features.
- Reduce regularization (smaller λ).
- Train for more epochs.
- Use a non-linear model.
- Perform feature engineering.

5) How to Fix Overfitting?

- Use more training data.
- Apply regularization (L1, L2, Elastic Net).
- Use dropout (in neural networks).
- Simplify the model (reduce complexity).
- Perform cross-validation.
- Use early stopping.
- Remove irrelevant features.
- Data augmentation (for images, text, etc.).

6) Techniques to Improve Generalization

Technique	Purpose	Common Use Cases
Regularization (L1/L2)	Adds penalty to loss	Linear models, Neural networks
Dropout	Randomly drops neurons	Deep learning
Early Stopping	Stop training at optimal point	Neural networks
Cross-Validation	Better performance estimate	All models
Feature Selection	Remove irrelevant features	All models
Data Augmentation	Increase training data	Image, text, audio
Ensemble Methods (Random Forest, etc.)	Reduce variance	Tree-based models

7) Key Takeaways

- Underfitting → high bias → model too simple → high error.
- Overfitting → high variance → model too complex → low training error but high test error.
- Use the right model complexity and regularization to get good generalization.
- Always evaluate the model on unseen data (validation/test set).
- Techniques like cross-validation, regularization, and early stopping help in building robust models.

Rahul Yadav

15. Bias-Variance Tradeoff

1) What is Bias-Variance Tradeoff ?

- Bias-Variance Tradeoff explains the balance between model simplicity and complexity.
- Bias is the error due to wrong assumptions (simpler model).
- Variance is the error due to high sensitivity to training data (complex model).
- We need to find the right balance to minimize the total error and get the best generalization on unseen data.

Decomposition of Error

$$E\left[(y - \hat{f}(x))^2\right] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

- Bias^2 : Error due to incorrect assumptions.
- Variance : Error due to sensitivity to training data.
- σ^2 : Irreducible error (noise in data).
- Total Error : Sum of all three components.

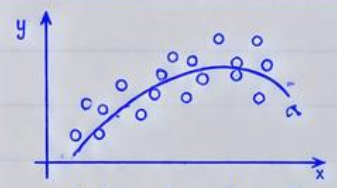
2) Intuition with Diagrams

High Bias (Underfitting)



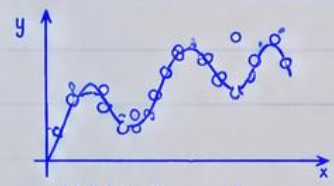
- Model is too simple.
- Cannot capture the actual pattern.
- High bias, low variance.
- High training and test error.

Good Fit (Right Balance)



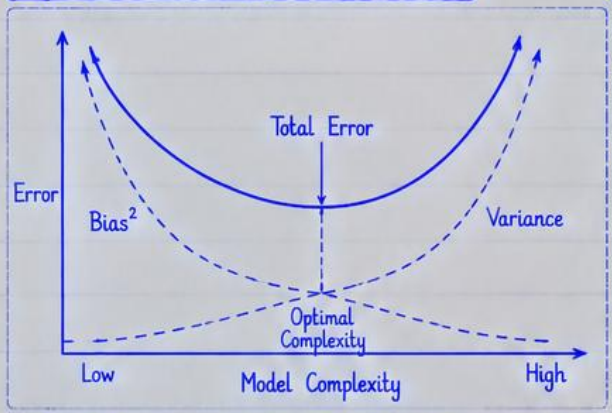
- Model captures the pattern well.
- Low bias, low variance.
- Low training and test error.
- Best generalization.

High Variance (Overfitting)



- Model is too complex.
- Learns noise from training data.
- Low bias, high variance.
- Low training error but high test error.

3) Bias-Variance Tradeoff Curve



4) Examples of Algorithms

Algorithm	Bias	Variance	Tendency
Linear Regression	High	Low	Underfitting
Ridge Regression	Medium	Low	Good Balance
Lasso Regression	Medium	Low	Good Balance
Decision Tree	Low	High	Overfitting
Random Forest	Low	Low	Good Balance
k-NN (small k)	Low	High	Overfitting
k-NN (large k)	High	Low	Underfitting
Neural Networks (large model)	Low	High	Overfitting

5) Techniques to Handle Bias-Variance Tradeoff

Technique	Reduces	How it Helps
Increase model complexity	Bias ↓	Adds more parameters/features
Regularization (L1/L2)	Variance ↓	Penalizes large weights
More training data	Variance ↓	Gives stable estimates
Feature selection	Variance ↓	Removes irrelevant features
Ensemble methods (Bagging, Random Forest)	Variance ↓	Averages multiple models
Early stopping	Variance ↓	Prevents overfitting in neural networks
Dropout	Variance ↓	Randomly drops neurons

6) Real-World Examples

- Linear Regression on few features → Underfitting (high bias).
- Deep Neural Network on small dataset → Overfitting (high variance).
- Random Forest → Good balance for many real-world problems.
- Increasing dataset size usually reduces variance.
- Regularization (L1/L2) helps to reduce variance without increasing bias much.

7) Key Takeaways

- Underfitting → high bias, model is too simple, high training and test error.
- Overfitting → high variance, model is too complex, low training error but high test error.
- The best model has a good balance between bias and variance.
- Use techniques like regularization, more data, feature selection and ensemble methods to improve generalization.
- Always evaluate on unseen data (validation/test set) to find the optimal model complexity.

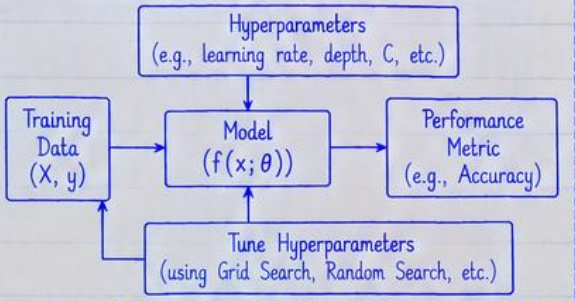
Rahul Yadav

16. Hyperparameter Tuning

1) What are Hyperparameters ?

- Hyperparameters are configuration settings used to control the learning process of a machine learning model.
- They are set before training and are not learned from the data.
- The right choice of hyperparameters can significantly improve model performance.
- Common hyperparameters: learning rate, number of trees, regularization strength, batch size, etc.

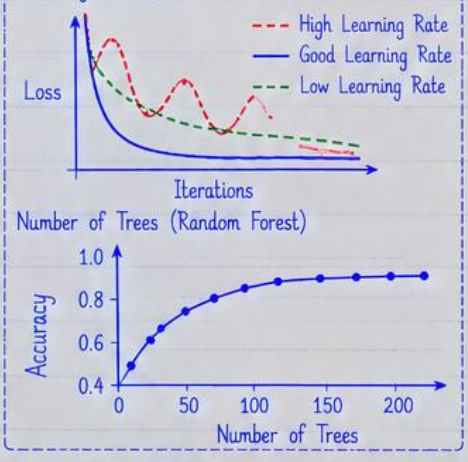
Model Training with Hyperparameters



2) Common Hyperparameters (by Algorithm)

Algorithm	Important Hyperparameters
Linear Regression	learning rate (for GD), regularization (L1, L2)
Logistic Regression	C (regularization strength), penalty (l1, l2)
Decision Tree	max_depth, min_samples_split, min_samples_leaf
Random Forest	n_estimators, max_depth, min_samples_split
Gradient Boosting (XGBoost, LightGBM)	learning_rate, n_estimators, max_depth, subsample, colsample_bytree
Support Vector Machine	C, kernel, gamma
K-Nearest Neighbors	n_neighbors, metric, weights
Neural Networks	learning rate, batch size, number of layers, dropout rate, number of epochs

Effect of Hyperparameters (Example)



3) Hyperparameter Tuning Techniques

a) Manual Tuning

- Try different values manually based on experience.
- Good for small problems.
- Time-consuming.
- Not efficient for large search spaces.

b) Grid Search

- Tries all possible combinations of hyperparameters.
- Exhaustive search.
- Guarantees the best combination from the given set.
- Computationally expensive.

c) Random Search

- Randomly samples combinations of hyperparameters.
- More efficient than Grid Search.
- Often finds a good model with fewer iterations.
- Works well when only a few hyperparameters are important.

d) Bayesian Optimization

- Uses past results to choose better hyperparameters.
- More efficient than Grid and Random Search.
- Common libraries: Optuna, Hyperopt, Scikit-Optimize.
- Works well for complex models.

4) Cross-Validation for Hyperparameter Tuning

- Use k-fold cross-validation to evaluate each hyperparameter combination.
- It helps to get a reliable performance estimate and avoids overfitting.
- The best hyperparameters are selected based on the average validation score.

K-Fold Cross-Validation (k = 5)

Fold 1	Train	Train	Train	Train	Validate
Fold 2	Train	Train	Train	Validate	Train
Fold 3	Train	Train	Validate	Train	Train
Fold 4	Train	Validate	Train	Train	Train
Fold 5	Validate	Train	Train	Train	Train

5) Example: GridSearchCV in Scikit-Learn

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5, 10]
}
model = RandomForestClassifier()
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

6) Key Takeaways

- Hyperparameters control the learning process and are not learned from data.
- Use cross-validation to evaluate hyperparameters.
- Grid Search is exhaustive but expensive.
- Random Search is more efficient and often works well.
- Bayesian Optimization is a powerful technique for complex models.
- Different algorithms have different important hyperparameters.
- The right hyperparameters can significantly improve model performance and generalization.

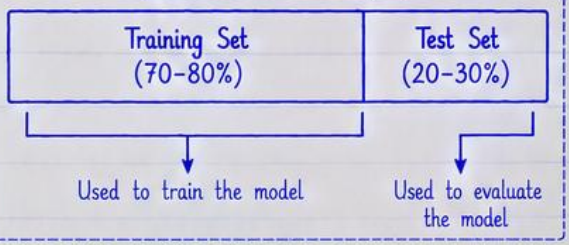
Rahul Yadav

17. Model Evaluation Strategies

1) Why Different Evaluation Strategies ?

- A single train-test split may give misleading results depending on the data.
- Evaluation strategies help us get a more reliable estimate of model performance.
- They reduce the risk of overfitting and give better generalization.
- Different strategies are useful for different dataset sizes and problem types.

Basic Train-Test Split



2) Common Evaluation Strategies

a) Holdout Method (Train-Test Split)

Train (70–80%)	Test (20–30%)

- Simplest method.
- Split data once into train and test sets.
- Fast and easy to use.
- May give high variance result depending on the split.

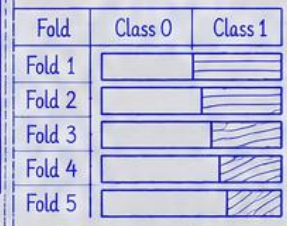
b) K-Fold Cross-Validation

Fold1	Fold2	Fold3	Fold4	Fold5
Train	Train	Train	Train	Validate
Train	Train	Train	Validate	Train
Train	Train	Validate	Train	Train
Train	Validate	Train	Train	Train
Validate	Train	Train	Train	Train

- Split data into K equal parts.
- Train on K-1 folds and validate on the remaining fold.
- Repeat K times and take the average result.
- Commonly used (K = 5 or 10).

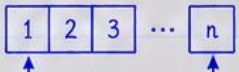
c) Stratified K-Fold

Each fold has same class distribution.



- Used for classification problems with imbalanced data.
- Maintains the same class distribution in each fold.

d) Leave-One-Out Cross-Validation (LOOCV)



Use 1 sample for validation and rest for training

- Special case of K-Fold where K = number of samples.
- Train on n-1 samples and test on 1 sample.
- Computationally expensive.
- Useful for very small datasets.

3) Train-Validation-Test Split

Training Set (60–70%)	Validation Set (10–20%)	Test Set (10–20%)

- Training Set: Used to train the model.
- Validation Set: Used to tune hyperparameters and select the best model.
- Test Set: Used to evaluate the final performance (unseen data).

When to Use What ?

Strategy	When to Use
Train-Test Split	Large datasets, quick evaluation
K-Fold Cross-Validation	Small to medium datasets
Stratified K-Fold	Classification with imbalanced classes
Leave-One-Out (LOOCV)	Very small datasets
Train-Validation-Test Split	When tuning hyperparameters

4) Time Series Cross-Validation (Walk-Forward)



- Used for time series data.
- Maintains the time order of data.
- Trains on past data and validates on future data.
- Prevents data leakage from future to past.

5) Evaluation Pipeline



6) Important Considerations

- Always evaluate on unseen data.
- Choose the right strategy based on data size and type.
- Use stratified splits for imbalanced classification.
- For time series, always maintain chronological order.
- Use cross-validation to get stable and reliable results.
- Combine evaluation metrics (e.g., Accuracy, Precision, Recall, F1-Score, ROC-AUC).

7) Key Takeaways

- Different evaluation strategies give different insights.
- K-Fold cross-validation is the most commonly used.
- Stratified K-Fold is important for imbalanced data.
- Use time series validation for sequential data.
- Always choose the strategy based on your problem and data characteristics.
- A good evaluation strategy helps build more reliable and generalizable models.

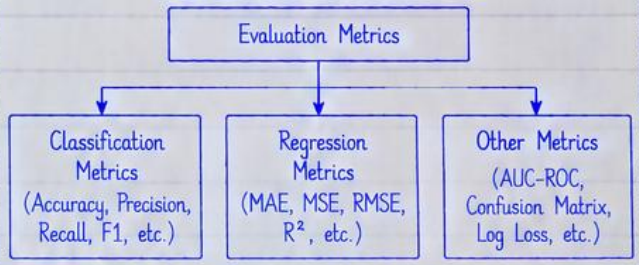
Rahul Yadav

18. Model Performance Metrics

1) Why Model Evaluation Metrics ?

- Metrics help us measure how well the model is performing.
- They give a clear and objective way to compare different models.
- The right metric depends on the type of problem (classification, regression, etc.).
- Using multiple metrics gives a better understanding of model performance.

Types of Evaluation Metrics



2) Classification Metrics (Binary / Multi-class)

Confusion Matrix (Binary Classification)

	Actual Positive (1)	Actual Negative (0)
Predicted Positive (1)	True Positive (TP)	False Positive (FP)
Predicted Negative (0)	False Negative (FN)	True Negative (TN)

Common Metrics

Metric	Formula
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$
Recall (Sensitivity)	$TP / (TP + FN)$
F1-Score	$2 \times (Precision \times Recall) / (Precision + Recall)$
Specificity	$TN / (TN + FP)$

When to Use ?

- Accuracy: Good for balanced datasets.
- Precision: Important when false positives are costly (e.g., spam detection).
- Recall: Important when false negatives are costly (e.g., disease detection).
- F1-Score: Useful for imbalanced datasets.
- Specificity: Used in medical testing and anomaly detection.

3) Multi-class Classification Metrics

- For multi-class problems, metrics are calculated for each class and then averaged.
- Two types of averaging:
  - Macro Average: Treats all classes equally.
  - Weighted Average: Considers class imbalance.
- Common metrics: Accuracy, Precision, Recall, F1-Score (macro/weighted).

Multi-class Confusion Matrix (3 classes)

	Pred A	Pred B	Pred C
Actual A	50	2	3
Actual B	4	45	6
Actual C	1	7	48

Averaging Methods

- Macro Average:
  - Give equal weight to all classes.
- Weighted Average:
  - Give weight based on support (number of samples) in each class.

4) Regression Metrics

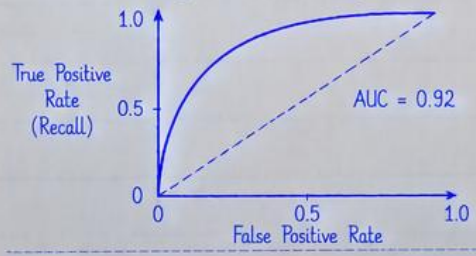
Metric	Formula	Description
MAE (Mean Absolute Error)	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Average of absolute errors. Less sensitive to outliers.
MSE (Mean Squared Error)	$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Squares the errors. Penalizes large errors.
RMSE (Root Mean Squared Error)	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$	Square root of MSE. Same unit as target variable.
R^2 Score (Coefficient of Determination)	$1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{\hat{y}}_i)^2}$	Value between 0 and 1. Higher is better.

When to Use ?

- MAE: Use when you want a simple and interpretable metric.
- MSE / RMSE: Use when large errors should be penalized more.
- R^2 Score: Use to understand how much variance is explained by the model.

5) Receiver Operating Characteristic (ROC) Curve

- Shows the trade-off between True Positive Rate (Recall) and False Positive Rate.
- Area Under Curve (AUC) measures the model's ability to distinguish between classes.
- AUC value ranges from 0 to 1 (higher is better).



6) Log Loss (Cross Entropy Loss)

- Measures the probability of correct class predictions.
- Used for classification problems.
- Lower value is better.
- Penalizes confident but wrong predictions more heavily.

$$\text{Log Loss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right]$$

7) Key Takeaways

- There is no single best metric for all problems.
- Choose the right metric based on the problem type and business requirement.
- Use multiple metrics to get a complete picture.
- For imbalanced data, prefer Precision, Recall, F1-Score, AUC-ROC.
- Always validate the model on unseen data using proper evaluation strategy.

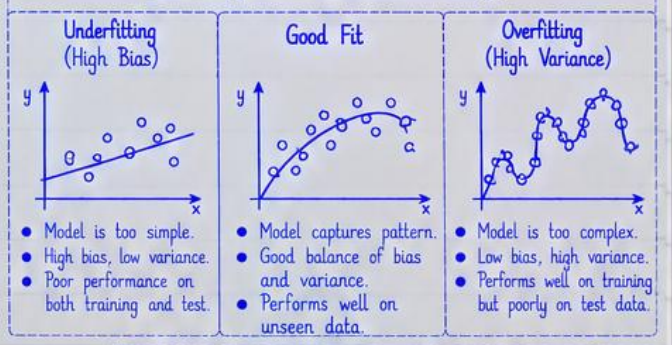
Rahul Yadav

19. Avoiding Overfitting and Underfitting

1) What is Overfitting and Underfitting ?

- Overfitting: Model learns training data too well, including noise, and performs poorly on new data.
- Underfitting: Model is too simple and cannot capture the underlying pattern in the data.
- The goal is to find the right balance so that the model generalizes well to unseen data.
- We can detect overfitting and underfitting by comparing training and validation performance.

Underfitting vs Good Fit vs Overfitting



- Model is too simple.
- High bias, low variance.
- Poor performance on both training and test.

- Model captures pattern.
- Good balance of bias and variance.
- Performs well on unseen data.

- Model is too complex.
- Low bias, high variance.
- Performs well on training but poorly on test data.

2) Comparison Table

Aspect	Underfitting (High Bias)	Good Fit	Overfitting (High Variance)
Model Complexity	Too Simple	Just Right	Too Complex
Training Error	High	Low	Very Low
Validation/Test Error	High	Low	High
Bias	High	Moderate	Low
Variance	Low	Moderate	High
Examples	Linear model on complex data	Well tuned model	Deep tree on small dataset

3) Techniques to Avoid Overfitting

Technique	How it Helps
More Training Data	Helps model learn general patterns
Cross-Validation	Checks generalization performance
Regularization (L1/L2)	Penalizes large weights
Dropout (Neural Networks)	Randomly drops neurons
Early Stopping	Stops training when validation error increases
Feature Selection	Removes irrelevant features
Data Augmentation	Increases training data artificially
Ensemble Methods	Combines multiple models (e.g., Random Forest, Boosting)

4) Techniques to Fix Underfitting

Technique	How it Helps
Increase Model Complexity	Use more complex model (e.g., deeper neural network, more trees)
Add More Features	Provide more informative features
Reduce Regularization	Use smaller regularization strength
Train for More Epochs	Allow model to learn better
Use Better Features	Create more meaningful features
Try Different Algorithms	Use algorithms with higher capacity

5) Regularization (L1 and L2)

- Regularization adds a penalty to the loss function to prevent overfitting by keeping model weights small.
- **L1 Regularization (Lasso)**: Adds absolute value of weights. Helps in feature selection (makes some weights zero).

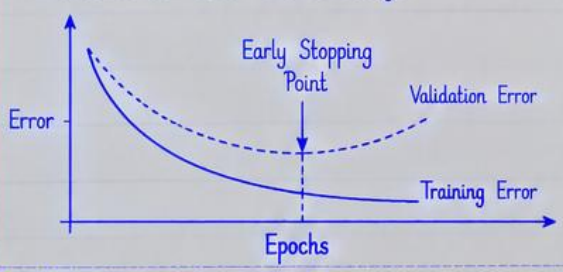
$$J(\theta) = \text{Loss} + \lambda \sum_{i=1}^{n} |\theta_i| \quad \text{(L1)}$$

- L2 Regularization (Ridge): Adds square of weights. Keeps all weights small but rarely zero.

$$J(\theta) = \text{Loss} + \lambda \sum_{i=1}^{n} \theta_i^2 \quad \text{(L2)}$$

6) Early Stopping

- Monitor validation error during training.
- Stop training when validation error starts increasing.
- Prevents the model from overfitting.



7) Cross-Validation (K-Fold)

- Split data into K equal parts (folds).
- Train on K-1 folds and validate on the remaining fold.
- Repeat K times and take the average performance.
- Provides a more reliable estimate of model performance.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iteration 1	Val	Train	Train	Train	Train
Iteration 2	Train	Val	Train	Train	Train
Iteration 3	Train	Train	Val	Train	Train
Iteration 4	Train	Train	Train	Val	Train
Iteration 5	Train	Train	Train	Train	Val

8) Key Takeaways

- Underfitting = high bias, model is too simple.
- Overfitting = high variance, model is too complex.
- Use techniques like regularization, cross-validation, early stopping, and more data to avoid overfitting.
- Increase model complexity or use better features to fix underfitting.
- The best model generalizes well to unseen data (good balance of bias and variance).

Rahul Yadav